

Audit cryptographique — `anibaledel/livreedhermes`

Dépôt audité <https://github.com/anibaledel/livreedhermes>

Passage 1 commit `6ce3985` (2026-09-10, « SPN 5 couches double référent »)

Passage 2 commit `16539cc` (2026-09-10, après refonte hybride géométrique + AEAD)

Passage 3 commit `d590010` (2026-09-11, durcissement `_decrypt/_xchacha_dec`) — cible + `3b7af6a8`
(achèvement renommage LH-5, sans effet crypto)

Périmètre `disk/disk_lib.py`, `stegano/stegano_lib.py`, `stegano/carter.py`, `encodeur.html` (stégano, vault, échange X25519), worker Cloudflare `worker/src/index.js`, scripts de gating

Méthode Revue manuelle + vérification empirique (scripts rejoués sur le code, Python 3 et Node 22)

Hors
périmètre Contenu éditorial, SEO, brevet FR2865054, prépublication IACR citée dans les en-têtes (non consultée)

Correctifs Fusionnés dans le fork : <https://github.com/CyberMind-FR/livreedhermes/pull/2>

1. Résumé exécutif

Ce rapport couvre deux passages. Le premier portait sur des primitives « maison » présentées comme supérieures à AES-256. Le dépôt a été refondu le jour même en architecture hybride : couche géométrique cantonnée à la diversification de clé, chiffrement confié à des primitives standard. Le second passage porte sur cette refonte.

État après refonte. La confidentialité et l'intégrité reposent désormais sur ChaCha20-Poly1305, AES-256-GCM, HKDF, X25519 et PBKDF2, c'est-à-dire sur des primitives éprouvées et non plus sur la construction géométrique. Les affirmations d'espace de clés fantaisistes ont disparu du code au profit de mentions honnêtes. Les treize constats critiques et élevés du premier passage sont corrigés, à une exception documentaire près.

Ce que le second passage a trouvé. Cinq défauts subsistaient ou avaient été introduits par la refonte, dont deux sérieux :

- Le durcissement du KDF du chiffrement disque était **entièrement contournable** : le MAC global était calculé avec la clé maître brute et vérifié avant toute dérivation, ramenant le coût d'un essai de passphrase hors ligne de 203 ms à 5,35 µs.
- Les cellules porteuses de la stéganographie étaient **repérables sans aucune clé** : les octets étaient écrits en

nibbles, donc dans [0..15], dans une grille de bruit couvrant [0..43].

S'y ajoutaient une perte de message à la capacité annoncée, un vault dont la clé maître était dérivée sans sel, et une clé privée X25519 exportable sans raison.

Correctifs. Les cinq sont corrigés et vérifiés par exécution, fusionnés dans le fork CyberMind-FR. Trois points restent ouverts côté auteur : le sel fixe de la grille Carter, la permutation Ref256 dont l'entropie réelle est huit fois plus faible qu'annoncé, et le débit.

Gravité résiduelle après correctifs : **Faible**. La couche géométrique n'est plus sur le chemin critique de la sécurité, ce qui était la recommandation principale du premier passage.

2. Passage 1 — constats et état actuel

Constats relevés sur 6ce3985, avec leur état sur 16539cc et après correctifs.

#	Constat initial	Gravité	État
D1	Permutation Ref256 constante : 12 288 configurations → 1 seule permutation (offsets calculés puis ignorés)	Critique	Partiel — offsets utilisés, mais 288 permutations distinctes seulement (cf. §4.6)
D2	Diffusion incomplète après 4 tours ; 19 chunks sur 84 diffusent complètement	Critique	Sans objet — le SPN ne chiffre plus, il dérive une clé
D3	Queue de secteur en two-time pad malléable	Critique	Corrigé — plus de flux XOR, AEAD sur le secteur entier
D4	Aucune authentification	Élevée	Corrigé — Poly1305 par secteur + HMAC global
D5	Chiffrement déterministe, ECB au niveau du chunk	Élevée	Corrigé — sel et nonce aléatoires par fichier
D6	Sel PBKDF2 codé en dur	Élevée	Corrigé — sel aléatoire, <code>passphrase_to_key</code> le retourne
D7	<code>security_stats()</code> annonçait $2^{159\,432}$ bits	Moyenne	Corrigé — remplacé par <code>description()</code> , sans compteur de bits
D8	PBKDF2 comme PRF de tour, 3,5 Ko/s	Moyenne	Amélioré — 44 Ko/s (cf. §4.8)
D9	<code>MixBlock</code> triangulaire sans diffusion inter-	Élevée	Corrigé — ShiftRows + MixColumns MDS

#	Constat initial	Gravité	État
	groupes		
D10	Ref360 : 342 entrées pour 116 permutations	Faible	Subsiste — documenté
S1	Message stocké en clair dans la grille	Critique	Corrigé — AEAD avant dissimulation
S2	Blocs 6×6 cassables en 4096 essais sans clé	Critique	Corrigé — conséquence de S1
S3	Clé A redondante avec Clé 2	Critique	Corrigé — Clé A retirée
S4	Distingueur 0/44 entre message et bruit	Élevée	Remplacé par un distingueur plus fort (cf. §4.2)
S5	PRNG non cryptographiques	Élevée	Corrigé — <code>secrets</code> et <code>crypto.getRandomValues</code>
S6	Clé A sur 31 bits	Élevée	Sans objet — Clé A retirée
S7	« Fausse lecture plausible » non vérifiée	Moyenne	Corrigé — échec explicite par key commitment
S8	Chemins des référents cassés	Faible	Corrigé — <code>_find_ref()</code>
W1..W5, G1	Worker Stripe et gating	Faible à Moyenne	Inchangés (cf. §5)

3. Architecture actuelle

Chiffrement disque. Passphrase → PBKDF2-SHA256 300 000 itérations → clé géométrique. Par secteur de 512 octets : la couche géométrique (S-box GF(2⁸), permutation Ref256, MixColumns MDS, permutation Ref360, 4 tours) dérive une clé de session, puis ChaCha20-Poly1305 chiffre et authentifie le secteur. Un HMAC-SHA256 couvre l'ensemble. Format : sel 32 octets, nonce global 24 octets, secteurs authentifiés, MAC 32 octets.

Stéganographie. Le message est chiffré (XChaCha20-Poly1305 par extension de nonce HKDF) et engagé par un HMAC de commitment, puis dissimulé aux positions désignées par les clés B, C et 2. La confidentialité ne dépend plus de la géométrie.

Navigateur. `encodeur.html` substitue AES-256-GCM à ChaCha20, absent de WebCrypto, et implémente en natif X25519, HKDF et PBKDF2. Le vault chiffre par entrée, avec manifest chiffré et HMAC global.

Cette séparation est saine : la construction géométrique reste originale et visible, sans porter la charge de la sécurité.

4. Passage 2 — constats sur 16539cc

4.1 Le KDF de 300 000 itérations était contournable (critique, corrigé)

```
mac = _hmac.new(master_key, payload, hashlib.sha256).digest()
```

Le MAC global était keyé par la clé maître brute et vérifié **avant** toute dérivation. Un attaquant tenant un fichier chiffré testait donc les passphrases contre le MAC sans jamais exécuter PBKDF2.

Chemin d'attaque	Coût par essai
------------------	----------------

via le MAC global	5,35 μ s
-------------------	--------------

via PBKDF2 300 000	203 ms
--------------------	--------

Facteur d'environ 38 000 sur une attaque par dictionnaire hors ligne : le durcissement ne servait à rien. Vérifié en retrouvant la passphrase d'un fichier de test par simple recalcul du HMAC.

Correctif : clé de MAC dédiée dérivée par HKDF de la sortie du KDF, et KDF appliqué avant la vérification. Coût par essai mesuré après correctif : 210 ms.

4.2 Les cellules porteuses étaient repérables sans clé (élevée, corrigé)

Le premier passage signalait un distingueur 0/44. La refonte l'a supprimé mais en a introduit un plus fort : les octets étaient écrits en nibbles, donc dans `[0..15]`, alors que le bruit couvre `[0..43]`. Mesures sur une grille 60×60 :

- les 226 cellules porteuses avaient toutes une valeur ≤ 15 , aucune au-dessus ;
- toute cellule > 15 était prouvablement du bruit, ce qui élimine d'emblée 64 % de la grille ;
- une forme dont les 6 cellules valent ≤ 15 n'a qu'une chance sur 432 d'être du bruit. Sur les 12 premiers blocs testés, les 12 blocs porteurs ressortaient.

Le message restait chiffré, mais sa présence et son emplacement étaient détectables sans clé, ce qui est l'inverse du but d'un système stéganographique. La note d'en-tête annonçant « pas de distingueur trivial » ne visait que l'ancien tell.

Correctif : encodage base-44 avec rembourrage aléatoire, écart à l'uniformité inférieur à 2^{-64} . Après correctif, sur 60 grilles, les symboles porteurs couvrent 44 valeurs sur 44 avec $\chi^2 = 45,1$ pour 43 degrés de liberté, seuil à 5 % de 59,3. Effet de bord favorable : 1,47 symbole par octet au lieu de 2.

4.3 Perte de message à la capacité annoncée (élevée, corrigé)

`max_message_len` comptait 32 octets de surcoût là où le format en consomme 72, soit 32 de commitment, 24 de nonce et 16 de tag. Un message de la taille annoncée passait l'encodage, était tronqué faute de positions, puis rejeté au décodage :

capacité annoncée : 268

ÉCHEC au décodage d'un message pourtant accepté : Positions insuffisantes : $600 < 688$

Correctif : capacité calculée depuis le format réel. Un message à la capacité maximale fait l'aller-retour, un caractère de plus est refusé à l'encodage.

4.4 Le vault dérivait sa clé maître sans sel (élevée, corrigé)

```
salt: new TextEncoder().encode('SecuBox-Vault-KDF-v1')
```

Ce libellé est identique pour tous les vaults et tous les utilisateurs. Une seule table précalculée passphrase vers clé maître ouvrirait n'importe quel fichier, quel que soit le nombre d'itérations. Le format embarquait pourtant déjà un sel aléatoire de 32 octets dans son en-tête, à l'offset 8, utilisé pour les sous-clés HKDF. La justification inscrite dans le code, éviter de conserver un sel séparé, ne tenait donc pas.

Correctif : version 2 du format, PBKDF2 salé par le sel de l'en-tête, lu avant la dérivation. Les vaults v1 restent lisibles et repassent en v2 au prochain export. Vérifié sous Node : création, export, relecture, rejet d'une mauvaise passphrase, et deux vaults créés avec la même passphrase donnent bien des clés maîtres distinctes.

4.5 Clé privée X25519 exportable (faible, corrigé)

`generateKey({name: 'X25519'}, true, ...)` rendait la clé privée éphémère exportable. Le drapeau ne concerne que la clé privée, la clé publique restant exportable par spécification : `false` ne change rien à l'échange.

4.6 Permutation Ref256 : 288 permutations, pas 12 288 (moyenne, ouvert)

Les offsets de quadrant sont désormais utilisés, ce qui corrige le bug du premier passage. Mais la table de 12 288 entrées ne produit que **288 permutations distinctes**, soit environ 8 bits au lieu des 13,6 attendus. Cause : les positions des 256 formes du référent sont déjà triées par ordre croissant, et `_perm_from_seq` trie par position absolue. L'identité de la forme est donc effacée, seul l'ordre des quadrants compte.

```
perms distinctes par ordre de quadrant : 12 | nombre d'ordres : 24  
formes dont les positions sont déjà triées : 256/256
```

Rendre la forme signifiante suppose de revoir la construction de la permutation, ce qui relève de l'auteur. Corrigé en documentation seulement. Sans conséquence sur la sécurité effective, la couche n'étant plus qu'une diversification de clé.

4.7 Nommage de l'algorithme (faible, corrigé en documentation)

`disk_lib.py` annonçait XChaCha20-Poly1305 et un nonce de 192 bits alors qu'il utilise ChaCha20Poly1305 de cryptography, soit le variant IETF à nonce de 96 bits. Sans conséquence pratique, le nonce par secteur étant dérivé par HKDF d'un nonce global de 192 bits et la clé changeant à chaque secteur, mais l'écart de nommage nuit à la

relecture. `stegano_lib.py` implémente de son côté une extension de nonce par HKDF, saine mais non conforme à XChaCha20 standard : les deux implémentations ne sont pas interopérables avec une bibliothèque tierce.

4.8 Débit (moyenne, ouvert)

44 Ko/s sur le chiffrement disque, contre 3,5 Ko/s avant refonte. Le coût est dominé par la reconstruction de 8 S-box par secteur dans la couche géométrique. Un cache de S-box par tour, ou une dérivation unique par fichier, y remédierait sans changer la sécurité.

4.9 Grille Carter : sel de dérivation fixe (moyenne, ouvert)

Le mode Carter dérive sa clé maître depuis une passphrase avec le sel fixe `SecuBox-Carter-KDF-v1`, même défaut que §4.4. Corriger suppose d'embarquer un sel avec la grille, donc un changement de format laissé à l'auteur.

4.10 Échange X25519 non authentifié (information)

Un attaquant actif qui substitue les clés publiques pendant l'échange obtient une session valide des deux côtés. L'empreinte de session affichée par la page est la parade prévue, à condition d'être comparée hors bande. À énoncer explicitement dans l'interface. Par ailleurs les clés de session sont conservées en clair dans `localStorage`, ce que la page documente.

5. Worker Cloudflare et gating (inchangés depuis le passage 1)

Points corrects : `stripe.webhooks.constructEventAsync` avec secret dédié ; secrets hors dépôt ; prix Pro non lu depuis le client ; jetons `crypto.randomUUID()` ; `payment_status` vérifié.

- **W1** : `currency` vient du client et `STRIPE_MIN_AMOUNT_CENTS` est comparé sans conversion. Forcer `eur` ou une liste blanche côté worker.
- **W2** : les jetons n'ont pas de TTL, et les événements `charge.refunded` / `charge.dispute.created` ne sont pas traités. Prévoir expiration ou révocation.
- **W3** : préférer POST avec le jeton dans le corps ou un en-tête ; raccourcir le TTL de `session:<id>` et invalider après le premier `claim`.
- **W4** : refuser la requête si `ALLOWED_ORIGIN` n'est pas défini plutôt que `*` ; ne pas dériver `success_url` de l'en-tête `Origin`.
- **G1** : le verrou « Pro » est côté client et les fichiers sont publics, ce que le code documente. Toute promesse commerciale d'exclusivité est à relativiser.

6. Recommandations restantes

1. Reprendre la construction de la permutation Ref256 pour que la forme choisie influe réellement, ou assumer les 8 bits dans la documentation.
2. Embarquer un sel aléatoire dans le format de la grille Carter.

3. Mettre en cache les S-box de la couche géométrique pour le débit.
4. Énoncer dans l'interface que l'échange X25519 n'est pas authentifié et que la comparaison d'empreinte hors bande est nécessaire.
5. Durcir le worker Stripe selon W1 à W4.
6. Ajouter des tests de non-régression sur les propriétés vérifiées ici : uniformité des symboles porteurs, coût d'un essai de passphrase, unicité des sels de vault.
7. Faire relire la couche géométrique par un tiers avant toute revendication formelle. Elle n'est plus sur le chemin critique, ce qui rend cette relecture souhaitable mais non bloquante.

7. Reproduction

Python 3 avec `cryptography`, et Node 22 pour le vault.

- **Permutations distinctes** : `set(tuple(v[0]) for v in GeoSPN(...).pt256.values())` → 1 au passage 1, 288 au passage 2.
 - **Contournement du KDF** : recalculer `hmac.new(candidat, payload, sha256)` sur les octets d'un fichier chiffré privés de ses 32 derniers, et comparer au MAC.
 - **Distingueur stégano** : reconstituer les cellules porteuses depuis les clés, comparer leur maximum à celui du bruit, puis compter les blocs où une forme donne 6 valeurs ≤ 15 .
 - **Perte de message** : encoder un message de `max_message_len` caractères, puis le décoder.
 - **Vault** : dériver la clé maître avec deux sels distincts pour une même passphrase et comparer.
-

Passage 3 — 2026-09-11 (commit `d590010`, cible + `3b7af6a8`)

Note de méthode

Clone mono-commit (`d590010` « Harden `_decrypt` and `_xchacha_dec` »). Le diff `16539cc...d590010` n'est pas rejouable localement. Rapport des passages 1 & 2 relu intégralement. Environnement : Python 3.12.3, `cryptography` 41.0.7, `argon2-cffi` présent, `pytest` 7.4.4. **scipy absent** → 4 tests statistiques (χ^2 Carter) *skipped*. Un seul commit postérieur (`3b7af6a8`, achèvement du renommage LH-5) — sans effet crypto.

1. Résumé exécutif

À `d590010` la crypto est **saine**. Confidentialité/intégrité sur primitives standard uniquement (ChaCha20-Poly1305 / nonce-étendu-HKDF, AES-256-GCM navigateur, HKDF, PBKDF2, Argon2id, X25519) ; couche géométrique = diversification de clé, hors chemin critique. **Les deux constats sérieux du passage 2 sont corrigés, vérifiés par exécution** : KDF disque non contournable (192,9 ms/essai, MAC keyé par `HKDF(geo_key)` et non par la clé maître brute) ; symboles porteurs stégano uniformes sur `[0..43]` (χ^2 poolé $55,5 < 59,3$). 46 tests passent (4 skips `scipy`) ; `cryptanalyse_spn.py` : « toutes les propriétés annoncées sont tenues ». Deux défauts résiduels de gravité **Faible** : **N1** (symbole de poids faible de l'en-tête de longueur réduit à 11/44 pour un message de longueur fixe — non exploitable) et

N2 (CLI dérive la clé maître du vault avec un **sel fixe**, atténué par l'Argon2id par-vault). **Gravité résiduelle globale : Faible**, inchangée depuis le passage 2.

2. État des constats passés @ d590010

Passage 1 — disque (D) : D1 partiel/documenté (Ref256 = 288 perms, 8,17 bits, diversification de clé, sans impact) ; D2 sans objet (SPN ne chiffre plus) ; D3-D7 corrigés (AEAD secteur entier, Poly1305/secteur + HMAC global, sel/nonce aléatoires par fichier, `description()` sans compteur de bits) ; **D8 ouvert** (perf) ; D9 corrigé (branche MDS = 5) ; D10 subsiste/documenté (Ref360 164 perms).

Passage 1 — stégano/worker : S1-S8 corrigés (AEAD avant placement, clé A retirée, base-44 uniforme, `secrets/getRandomValues`, key-commitment HMAC, `_find_ref`) ; **W1 ouvert** (currency client, `worker/src/index.js:86`) ; **W2 partiel** (jeton sans TTL ; `refunded/dispute` non traités) ; W3-W4 corrigés (fenêtre de grâce ; `resolveOrigin` sur `allowlist`) ; G1 inchangé/documenté (gate client).

Passage 2 : §4.1 KDF contournable **corrigé** (192,9 ms, HMAC clé-brute invalide) ; §4.2/LH nibbles repérables **corrigé** (base-44 + rembourrage aléatoire ; χ^2 55,5, 44/44 valeurs) ; §4.3/LH-3 perte de message **corrigé** ; §4.4 vault sans sel (navigateur) **corrigé** (format v2 salé) ; §4.5 X25519 exportable (navigateur) **corrigé** (`extractable=false`) ; §4.6/Ref256 288 perms **ouvert/documenté** ; §4.7/LH-5 nommage XChaCha **corrigé (doc)** ; §4.8 débit **ouvert** ; §4.9 Carter sel fixe (navigateur) **corrigé** ; §4.10/LH-2 échange non authentifié **corrigé/amélioré** (X3DH navigateur + Python, vérif. fingerprint hors bande à assurer) ; LH-1 message hors alphabet **corrigé** (`ValueError`) ; LH-2 déni plausible v3 **corrigé** (revue seule) ; LH-4 en-tête de longueur non authentifié **corrigé** (HMAC sur `header||inner`) ; CR-1 capacité Carter Random **corrigé** (refus 30 car. = 0 %) ; CR-3 masques **corrigé (doc)** ; SPN-4 chunk 1 nul **corrigé (doc)** (relation linéaire détruite ~128 bits) ; DOC-1 **corrigé** (`cryptanalyse_spn.py` échoue non-nul si écart ; exécuté : propriétés tenues).

3 points ouverts du passage 2 : (1) sel fixe grille Carter → corrigé côté navigateur mais **réapparaît à la CLI = N2** ; (2) entropie Ref256 → **ouvert** (288 perms, 8,17 bits) ; (3) débit → **ouvert**.

3. Constats NOUVEAUX

N1 — En-tête de longueur : symbole de poids faible non uniforme (Faible). `crypto_core.py:92-103` : `u = data + span * randbelow(k), span = 2^(8 * len(b))`. Pour l'en-tête `span = 2^32 ≡ 4 (mod 44)`, `gcd(4, 44)=4` : le rembourrage ne parcourt que `{0, 4, ..., 40}` → symbole bas = **11 valeurs/44**. Empirique : message de longueur fixe (20 car.), 20 000 tirages → position 0 $\chi^2=60014$, 11/44 valeurs (les autres uniformes). Masqué sur longueurs variables (χ^2 en-tête poolé 52,5, 44/44) ; payload non affecté. Non exploitable en pratique, mais contredit la marge « $\leq 2^{-64}$ » annoncée pour tous les symboles. *Fix* : donner une entrée à entropie pleine au champ longueur (masque HKDF ou intégration au flux AEAD).

N2 — Vault CLI : clé maître dérivée avec sel fixe (Faible). `secu_box_cli.py:69` : `passphrase_to_key(pp, b'SecuBox-Vault-KDF-v1')` — sel constant. Vérifié : même passphrase → `master_key` identique. **Atténué** par Argon2id à sel aléatoire par-vault (`vault_lib.py`), donc clés finales/fichiers distincts. Impact : portion PBKDF2 300k précalculable (rainbow passphrase → `master_key`) ; corrélation si une `master_key` fuit. *Fix* : sel aléatoire (déjà dans

l'en-tête du vault), ou passer la passphrase directement à l'Argon2id.

Durcissement d590010 (revue) — sans régression. `_xchacha_dec` valide `len(data) ≥ 40` ; `_decrypt` borne `total_len ≤ 16 Mio`, refuse `< 32`, décode en `ascii` strict. Défensif, aucun défaut introduit.

4. Résultats des tests

- `test_regression.py` : **29 passés / 0 échec**.
- `test_statistical.py` : **17 passés, 4 skipped** (scipy absent).
- Mesures : avalanche clé grille 0,977 ; sensibilité clé XChaCha 0,501 ; avalanche message 0,977 ; entropie Carter 256/360/Mix 5,455/5,459/5,458 (max 5,4594) ; entropie bit XChaCha 0,9982 ; autocorr max_r 0,0178 < 0,0222 ; capacité Carter Random post-CR-1 refus 30 car. 0,0 %.
- `cryptanalyse_spn.py --seed 1` : « **Toutes les propriétés annoncées sont tenues** » (Ref256 288/8,17 bits, DDT ≤4 niveau AES, branche diff.=lin.=5, diffusion palier idéal à 4 tours, 0 point fixe, χ^2 sondage < seuil).

5. Verdict et recommandations

Verdict : à d590010 l'architecture hybride est saine ; tous les constats critiques/élevés des passages 1 & 2 corrigés + vérifiés. **Gravité résiduelle Faible** (N1, N2 sans exploitation pratique ; perf/entropie documentés).

Recommandations par priorité : 1. **N2** — sel aléatoire du vault au lieu du sel fixe `secu_box_cli.py:69`. 2. **N1** — entrée à entropie pleine pour le champ longueur (masque HKDF / flux AEAD). 3. **W1/W2** — `currency` en liste blanche côté worker ; traiter `charge.refunded/dispute` (révocation jeton). 4. **§4.6** — assumer 8 bits Ref256 (déjà en en-tête) ou revoir la construction. 5. **§4.8** — cacher les S-box par fichier/tour dans `_geo_derive` (débit). 6. **§4.10** — expliciter dans l'UI la vérification hors bande du fingerprint. 7. Tests de non-régression : uniformité du champ longueur (N1), unicité du sel de dérivation vault (N2).

Aucun fichier du dépôt cloné modifié ; vérifications empiriques via scripts hors arbre.

Passage 4 — 2026-09-11 (commit aede483, delta 5 commits sur 3b7af6a)

Note de méthode

`git fetch` de `anibaledel/livreedhermes` : **5 nouveaux commits** depuis l'état audité au passage 3 (3b7af6a) → HEAD aede483. Delta rejouable cette fois (`git show/git diff`). Environnement identique (Python 3.12.3, cryptography, argon2-cffi, pytest 7.4.4 ; **scipy absent** → tests statistiques sautés). Les 5 commits : - 4fe96da DOC-1 + CR-3 : correction des comptes Ref360 et du commentaire de masque. - 52bc2ee porte **Carter-18** et **Carter-Hybrid** (reconstruits sur le code du dépôt). - bf8b717 expose les fonctions de **session** Carter-18/Hybrid (`secu_box.py`). - f759431 couverture statistique Carter-18/Hybrid (`test_statistical.py`). - aede483 **Fix N2** : sel aléatoire par-vault pour la dérivation Argon2id.

1. Résumé exécutif

À aede483 la crypto reste **saine**. La grande nouveauté (Carter-18, Carter-Hybrid et leurs variantes de session X25519) est **hors chemin critique de confidentialité** : `encode_carter_18/encode_carter_hybrid` chiffrent d'abord par `_encrypt(message, xchacha_key)` (XChaCha20-Poly1305, AEAD vérifié), puis ne font que *placer* les symboles selon une grammaire dérivée par HKDF d'une clé et un référent géométrique **de graine publique**. La confidentialité/intégrité ne dépend jamais de la géométrie. **N2 est corrigé upstream** (aede483, vérifié). **DOC-1/CR-3** re-documentés correctement (masque additif = couche défensive, non source d'indiscernabilité, déterministe par `grammar_key`). Restent ouverts, non touchés par ce delta : **N1** (en-tête de longueur), **W1/W2** (worker Stripe), entropie Ref256 (§4.6), débit (§4.8). **Gravité résiduelle globale : Faible**, inchangée.

2. État des constats @ aede483

N2 — corrigé upstream. `secu_box_cli.py : _vault_key()` ne pré-dérive plus par PBKDF2 à sel fixe (b'SecuBox-Vault-KDF-v1') ; la passphrase est passée telle quelle (`pp.encode('utf-8')`) à l'Argon2id à **sel aléatoire par-vault** de `vault_lib`. Correctif juste. ⚠ **Sans repli de compatibilité** : les vaults créés avant le correctif (clé maître = PBKDF2 à sel fixe) ne s'ouvrent plus. Acceptable si aucun vault n'existe en production ; à signaler sinon. (La PR de contribution CyberMind-FR/livreedhermes#5 proposait un repli legacy — devenu redondant, à retirer au rebase.)

N1 — ouvert. `crypto_core.py` inchangé : le symbole de poids faible de l'en-tête de longueur reste à 11/44 valeurs pour une longueur fixe. Les nouveaux modes Carter-18/Hybrid routent par le même `payload_to_symbols()` → ils héritent du même défaut (non exploitable). Couvert par la contribution #5.

W1/W2 — ouverts. `worker/src/index.js` non touché par ce delta. Couverts par #5.

§4.6 (Ref256 8,17 bits), §4.8 (débit) — ouverts/documentés, inchangés.

DOC-1 / CR-3 — re-documentés (4fe96da). Le commentaire de masque additif précise désormais qu'il dérive de `grammar_key` **seul**, sans aléa propre à la grille (même `grammar_key` → mêmes masques à chaque appel), qu'il est conservé comme couche défensive et **ne doit pas être réemployé** ailleurs comme masque cryptographique générique. Exact et honnête ; les comptes Ref360 sont corrigés.

3. Constats NOUVEAUX (revue Carter-18 / Carter-Hybrid / session)

C18-1 — Masque additif déterministe par clé (Négligeable / informatif). `carter_random.py` : dans Carter-18 et Carter-Hybrid, `grid[gr][gc] = (nibbles[ni] + masks[ni]) % ALPHA_LEN` avec `masks = _derive_masks(grammar_key, ...)`. Le masque ne dépend que de `grammar_key` → **identique d'un message à l'autre** pour une même clé. **Sans danger ici** : `nibbles` sont les symboles d'un *payload déjà chiffré* (XChaCha20-Poly1305, nonce aléatoire), donc pseudo-aléatoires et indépendants à chaque message ; l'ajout d'un offset constant ne crée pas de réutilisation de flux (ce n'est pas un one-time-pad sur du clair). Vérifié : deux chiffrements du même message/clé différent sur ~7900/8100 cellules. Constat purement informatif ; upstream le documente déjà (C18/CR-3). *Recommandation* : ne jamais promouvoir ce masque en masque cryptographique autonome.

C18-2 — Alignement encode/decode conditionné à `grid_size % 18 == 0` (Robustesse, très faible).

`encode_carter_18/decode_carter_18` incrémentent `ni` pour chaque position d'une forme, y compris hors bornes (`if 0 <= gr < grid_size ...`). Pour la grille par défaut 90×90 (multiple de 18) toutes les positions sont dans les bornes : aucun impact. Pour une `grid_size` non multiple de 18, encode et decode sauteraient les mêmes positions mais l'invariant n'est pas garanti. *Recommandation* : `assert grid_size % BLOCK_18 == 0` en entrée (idem sous-blocs 6×6 pour Hybrid).

Session Carter (revue) — saine. `encode_carter_session_18/hybrid` (`secu_box.py`) utilisent `session_keys['steg_key']` (issu de `Session.derive()`, ECDH X25519 + HKDF) comme `master_key`, re-splitté par `_carter_split` en clé AEAD + clé grammaire. Hiérarchie de clés correcte, aucune clé longue-durée réutilisée en clair. Le mode Hybrid par bloc est dérivé de la **clé** et non de la longueur du message (choix explicite et correct : pas de distingueur géométrique court/long).

4. Résultats des tests

- Suite complète pytest @ aede483 : **50 passés / 6 skipped** (scipy absent — dont les 2 nouveaux tests χ^2 Carter-18/Hybrid).
- `py_compile` : `carter_random.py`, `secu_box.py`, `secu_box_cli.py` — OK.
- Fonctionnel (hors arbre) : Carter-18 round-trip OK (capacité 1234 car. sur 90×90), Carter-Hybrid round-trip OK (capacité 620 car.) ; **rejet d'une clé fausse par l'AEAD** (`ValueError`) pour les deux ; non-déterminisme confirmé (nonce aléatoire).

5. Verdict et recommandations

Verdict : à aede483, architecture toujours saine ; N2 corrigé upstream ; extension Carter-18/Hybrid/session bien conçue, géométrie hors chemin critique, aucun **nouveau** constat exploitable. **Gravité résiduelle : Faible**, inchangée.

Recommandations restantes (par priorité) : 1. **N1** — entrée à entropie pleine pour le champ longueur (contribution #5). 2. **W1/W2** — liste blanche de devises + révocation sur remboursement/litige (#5). 3. **N2** — si des vaults antérieurs existent en prod, prévoir un repli/migration (le correctif upstream aede483 n'en offre pas). 4. **C18-2** — assertion `grid_size % 18 == 0` dans Carter-18/Hybrid. 5. **§4.6 / §4.8** — entropie Ref256 assumée à 8 bits ; débit `_geo_derive`.

Aucun fichier du dépôt cloné modifié ; vérifications empiriques via scripts hors arbre.

Annexe A — Benchmark matériel réel (2026-09-11)

Mesures sur la box de production **gk2** (MOCHABin, Marvell **Armada 7040 / Cortex-A72**, aarch64, 4 cœurs), Python 3.11.2, cryptography 47.0.0. Micro-benchmarks mono-thread (`time.perf_counter`, warmup + boucle timée). But : chiffrer le point ouvert **§4.8 (débit)** sur l'architecture cible réelle.

A.1 — Cœur Hermes souverain (secubox_core.crypto, primitives OpenSSL C)

Opération	Débit	Latence
Identity.generate (X25519)	6 209 /s	161 µs
Identity.generate (X25519 + Ed25519)	3 420 /s	292 µs
Session.establish (ECDH X25519 + HKDF)	1 959 /s	510 µs
Ed25519 sign	8 148 /s	123 µs
Ed25519 verify	2 479 /s	403 µs
ChaCha20-Poly1305 chiffrement	60 MB/s @1 KiB · ~ 200 MB/s @≥16 KiB	8–17 µs (petits messages)

→ Adéquat pour l'usage identité/mesh : établissement de session sous la milliseconde, AEAD à ~200 MB/s. **Aucun goulot côté crypto réelle.**

A.2 — Couche Carter stégano (pure-Python) — message 65 car., grille 90×90

Mode	Encode	Decode	Capacité
baseline (AEAD + payload_to_symbols)	1 944 /s · 0,51 ms	—	—
Carter-Random-256	18 /s · 55,9 ms	140 /s · 7,2 ms	235 car.
Carter-18	21 /s · 47,7 ms	267 /s · 3,8 ms	1 185 car.
Carter-Hybrid	20 /s · 49,4 ms	255 /s · 3,9 ms	878 car.

A.3 — Décomposition du coût d'encodage (~50 ms)

Étape	Coût arm64
Init grille secrets.randbelow(44) × 8100 cellules	42,5 ms (≈ 85 %)
_derive_masks	1,0 ms
Build référent 18×18 (à froid, puis caché)	262 ms (1×/graine)

Étape	Coût arm64
-------	------------

Décodage (réfèrent déjà en cache)	~4 ms
-----------------------------------	-------

A.4 — Lecture §4.8 et suite donnée

Le coût d'encodage Carter est **quasi entièrement du remplissage CSPRNG cellule-par-cellule**, pas de la crypto. Il se réduit à ~0,5 ms ($\approx \times 25$) en tirant l'entropie de couverture en **un seul `os.urandom` en bloc + échantillonnage base-44 par rejet** — même source, même uniformité (χ^2 38,7 < 59,3, 44/44 valeurs). Correctif proposé à l'upstream : **PR CyberMind-FR/livreedhermes#6** (appliqué à Carter-Random/18/Hybrid ; Carter classique laissé inchangé car ses tests d'avalanche à bruit figé dépendent de la granularité de consommation d'`os.urandom`). Le build de réfèrent à froid (262 ms) reste amorti par le cache ; envisager un pré-chauffage au démarrage si la latence du premier message importe.

Conclusion débit : les primitives réelles sont rapides ; Carter reste adapté à de la **messagerie** (~8 ms/encode après optimisation, ~4 ms/decode), pas à du volume soutenu.